

Pattern CRUD Ereditabile - Guida alla Replicazione

Questa guida spiega come creare nuove gestioni CRUD basate sul pattern implementato per `ana_geo_capoluogo`.

Struttura del Pattern

Il sistema CRUD è composto da 4 elementi principali:

1. **Model** (ereditato da `BaseEntity`)
2. **Service** (ereditato da `BaseCrudService<T>`)
3. **Dialog Component** (modale per Insert/Edit)
4. **Page Component** (pagina con DataGrid)

Step 1: Creare il Model

Posizione: `/Models/NomeEntita.cs`

```
using System.ComponentModel.DataAnnotations;

namespace GestioneViaggi.Models;

/// <summary>
/// Rappresenta [descrizione entità] (tabella nome_tabella)
/// </summary>
public class NomeEntita : BaseEntity
{
    [Required(ErrorMessage = "Il campo è obbligatorio")]
    [StringLength(100, ErrorMessage = "Max 100 caratteri")]
    public string Nome { get; set; } = string.Empty;

    [StringLength(500, ErrorMessage = "Max 500 caratteri")]
    public string? Descrizione { get; set; }

    // Aggiungi altri campi necessari
}
```

Note:

- `BaseEntity` fornisce automaticamente la proprietà `Id` (non visibile all'utente)
- Usa `DataAnnotations` per validazione automatica

Step 2: Creare il Service

Posizione: `/Services/CRUD/NomeEntitaService.cs`

```
using GestioneViaggi.Models;
using GestioneViaggi.Services.Database;
using Microsoft.Extensions.Logging;
using Npgsql;

namespace GestioneViaggi.Services.CRUD;

public class NomeEntitaService : BaseCrudService<NomeEntita>
{
    // Configura nome tabella e colonna ID
    protected override string TableName => "nome_tabella_db";
    protected override string IdColumnName => "id_column_db";

    public NomeEntitaService(IDatabaseService databaseService,
        ILogger<NomeEntitaService> logger)
        : base(databaseService, logger)
    {
    }

    public override async Task<NomeEntita> CreateAsync(NomeEntita entity)
    {
        try
        {
            await using var connection = await
                _databaseService.GetConnectionAsync();
            var sql = @"
                INSERT INTO nome_tabella_db (campo1, campo2)
                VALUES (@campo1, @campo2)
                RETURNING id_column_db, campo1, campo2";

            await using var command = new NpgsqlCommand(sql, connection);
            command.Parameters.AddWithValue("campo1", entity.Campo1);
            command.Parameters.AddWithValue("campo2",
                (object?)entity.Campo2 ?? DBNull.Value);

            await using var reader = await command.ExecuteReaderAsync();
            if (await reader.ReadAsync())
            {
                return MapFromReader(reader);
            }

            throw new Exception("Impossibile creare l'entità");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Errore durante la creazione");
            throw;
        }
    }

    public override async Task<NomeEntita> UpdateAsync(NomeEntita entity)
    {
        try
```

```

    {
        await using var connection = await
_databaseService.GetConnectionAsync();
        var sql = @"
            UPDATE nome_tabella_db
            SET campo1 = @campo1, campo2 = @campo2
            WHERE id_column_db = @id
            RETURNING id_column_db, campo1, campo2";

        await using var command = new NpgsqlCommand(sql, connection);
        command.Parameters.AddWithValue("id", entity.Id);
        command.Parameters.AddWithValue("campo1", entity.Campo1);
        command.Parameters.AddWithValue("campo2",
(object?)entity.Campo2 ?? DBNull.Value);

        await using var reader = await command.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return MapFromReader(reader);
        }

        throw new Exception($"Entità con ID {entity.Id} non trovata");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Errore durante l'aggiornamento");
        throw;
    }
}

protected override NomeEntita MapFromReader(NpgsqlDataReader reader)
{
    return new NomeEntita
    {
        Id = ReadInt(reader, "id_column_db"),
        Campo1 = reader.GetString(reader.GetOrdinal("campo1")),
        Campo2 = ReadNullableString(reader, "campo2")
    };
}
}

```

Helper methods disponibili:

- `ReadInt(reader, "columnName")` - legge un int
- `ReadNullableString(reader, "columnName")` - legge una stringa nullable
- `GetAllAsync()` e `DeleteAsync()` sono già implementati in `BaseCrudService`

Step 3: Creare il Dialog Component

Posizione: `/Components/Shared/NomeEntitaDialog.razor`

```

@using GestioneViaggi.Models
@using MudBlazor

<MudDialog>
    <TitleContent>
        <MudText Typo="Typo.h6">
            @(IsEditMode ? "Modifica [Entità]" : "Nuova [Entità]")
        </MudText>
    </TitleContent>
    <DialogContent>
        <MudForm @ref="_form" Model="@Entity">
            <!-- PRIMO CAMPO OBBLIGATORIO: con asterisco rosso e uppercase
enforced -->
            <MudTextField @ref="_firstField"
                Value="@Entity.Campo1"
                ValueChanged="@HandleCampo1Changed"
                For="@(() => Entity.Campo1)"
                Label="Campo 1"
                Variant="Variant.Outlined"
                Immediate="true"
                Required="true"
                RequiredError="Campo 1 è obbligatorio"
                MaxLength="100"
                tabIndex="1"
                Class="mb-3 uppercase-input"
                Adornment="Adornment.End"
                AdornmentText="*"
                AdornmentColor="Color.Error"
                OnBlur="@(() => HandleFieldBlur(_firstField))"
                OnKeyDown="@HandleKeyDown" />

            <!-- SECONDO CAMPO OPZIONALE: senza asterisco -->
            <MudTextField @bind-Value="Entity.Campo2"
                For="@(() => Entity.Campo2)"
                Label="Campo 2 (opzionale)"
                Variant="Variant.Outlined"
                Lines="3"
                MaxLength="500"
                tabIndex="2"
                Class="mb-3" />

            <!-- Aggiungi altri campi con tabIndex progressivo (3, 4,
5...) -->
        </MudForm>
    </DialogContent>
    <DialogActions>
        <MudButton OnClick="Cancel" Variant="Variant.Text"
Color="Color.Default">
            Annulla
        </MudButton>
        <MudButton OnClick="HandleSubmit" Variant="Variant.Filled"
Color="Color.Primary">
            @(IsEditMode ? "Aggiorna" : "Crea")
    </DialogActions>

```

```

        </MudButton>
    </DialogActions>
</MudDialog>

@code {
    private MudForm? _form;
    private MudTextField<string>? _firstField;

    [CascadingParameter]
    private IMudDialogInstance? MudDialog { get; set; }

    [Parameter]
    public NomeEntita Entity { get; set; } = new NomeEntita();

    [Parameter]
    public bool IsEditMode { get; set; }

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender && _firstField != null)
        {
            // SetFocus automatico sul primo campo quando il dialog si
apre
            await _firstField.FocusAsync();
        }
    }

    private void Cancel()
    {
        MudDialog?.Cancel();
    }

    // ⚠ OBBLIGATORIO: Handler per uppercase in tempo reale
    private void HandleCampo1Changed(string value)
    {
        Entity.Campo1 = value?.ToUpper() ?? string.Empty;
        StateHasChanged(); // Forza re-render per mostrare uppercase
    }

    private async Task HandleFieldBlur(MudTextField<string>? field)
    {
        if (field != null)
        {
            await field.Validate();
            if (field.Error)
            {
                // Delay per permettere al click su "Annulla" di essere
processato prima del focus back
                await Task.Delay(150);
                try
                {
                    await field.FocusAsync();
                }
                catch
            }
        }
    }
}

```

```

        {
            // Ignora errori se il dialog è stato chiuso
        }
    }
}

private async Task HandleKeyDown(KeyboardEventArgs e)
{
    if (e.Key == "Enter")
    {
        await HandleSubmit();
    }
}

private async Task HandleSubmit()
{
    if (_form != null)
    {
        await _form.Validate();
        if (_form.IsValid)
        {
            MudDialog?.Close(DialogResult.Ok(Entity));
        }
        else
        {
            // Se ci sono errori, focus sul primo campo invalido
            // Nota: estendere la logica per altri campi se necessario
            if (_firstField?.Error == true)
            {
                await _firstField.FocusAsync();
            }
        }
    }
}
}

```

Tipi di input MudBlazor disponibili:

- **MudTextField** - testo semplice
- **MudNumericField** - numeri
- **MudDatePicker** - date
- **MudSelect** - dropdown
- **MudCheckBox** - checkbox

⚡ Pattern SetFocus e TAB Navigation:

1. **Primo campo:** Sempre con `@ref="_firstField"` e `tabindex="1"` (lowercase!)
2. **SetFocus automatico:** `OnAfterRenderAsync` chiama `_firstField.FocusAsync()` al primo render (con `Task.Delay(100)`)
3. **TAB Navigation:** Assegnare `tabindex` progressivo (1, 2, 3...) dall'alto verso il basso, da sinistra a destra

4. **Campo tipo:** Per campi non-string, usare `MudTextField<int>?`, `MudTextField<decimal>?`, etc.
5. **Layout multi-colonna:** Usare `tabindex` per definire ordine logico di navigazione
6. **IMPORTANTE:** MudBlazor richiede `tabindex` lowercase, non `TabIndex`

🌟 Campi Obbligatori - UX Best Practice:

1. **Asterisco rosso:** Usare `AdornmentText="*"` con colore rosso per indicare visivamente i campi obbligatori
2. **Required="true":** Attiva la validazione MudBlazor
3. **RequiredError:** Messaggio custom di errore quando il campo è vuoto
4. **Label opzionali:** Aggiungere "(opzionale)" nel label per campi non obbligatori
5. **Adornment:** Usare `AdornmentText="*"` con `AdornmentColor="Color.Error"`
6. **Focus Trap Intelligente:** Implementare `OnBlur` con delay per permettere il funzionamento del pulsante "Annulla" pur mantenendo il focus sul campo invalido
7. **🔴 Uppercase Enforced - OBBLIGATORIO per tutti i campi di testo:**
 - Usare `Value` / `ValueChanged` con metodo custom invece di `@bind-Value`
 - `ValueChanged="@HandleNomeChanged"` - chiama metodo che converte e forza re-render
 - `Immediate="true"` - applica la conversione immediatamente
 - **AGGIUNGERE** `Class="uppercase-input"` per visualizzare in tempo reale il testo maiuscolo mentre si digita
 - Nel @code creare metodo: `Entity.Nome = value?.ToUpper(); StateHasChanged();`
 - Il dato viene salvato in UPPERCASE nel database PostgreSQL
 - **ECCEZIONE:** Non applicare a campi Password o email case-sensitive
 - ⚠ **NON usare lambda inline o Converter:** non forzano re-render corretto

Esempio campo obbligatorio con uppercase:

```
<MudTextField Value="@Entity.Nome"
              ValueChanged="@HandleNomeChanged"
              Label="Nome"
              Immediate="true"
              Required="true"
              RequiredError="Il nome è obbligatorio"
              Class="uppercase-input"
              Adornment="Adornment.End"
              AdornmentText="*"
              AdornmentColor="Color.Error"
              OnBlur="@(() => HandleFieldBlur(_firstField))" />

// Nel @code:
private void HandleNomeChanged(string value)
{
    Entity.Nome = value?.ToUpper() ?? string.Empty;
    StateHasChanged(); // OBBLIGATORIO per mostrare uppercase in tempo
    reale
}
```

Esempio campo opzionale:

```
<MudTextField @bind-Value="Entity.Note"
              Label="Note (opzionale)" />
```

Campi Numerici - Formattazione Italiana con Separatore Migliaia:

REGOLA OBBLIGATORIA: Tutti i campi numerici (int, decimal) devono essere visualizzati con il separatore delle migliaia in formato italiano (punto come separatore migliaia, virgola come separatore decimali).

1. **In Input (Dialog):** Usare **MudNumericField** con **Culture** italiano
2. **In Grid (Visualizzazione):** Usare **PropertyColumn** con **Format** o **CellTemplate** per formattazione custom

Esempio campo numerico in Dialog:

```
<MudNumericField @bind-Value="Entity.Residenti"
                 For="@(() => Entity.Residenti)"
                 Label="Residenti - opzionale"
                 Variant="Variant.Outlined"
                 Min="0"
                 Culture="@ (new System.Globalization.CultureInfo("it-IT"))"
                 Format="N0"
                 tabIndex="5"
                 Class="mb-3" />

<MudNumericField @bind-Value="Entity.Superficie"
                 For="@(() => Entity.Superficie)"
                 Label="Superficie (kmq) - opzionale"
                 Variant="Variant.Outlined"
                 Min="0"
                 Culture="@ (new System.Globalization.CultureInfo("it-IT"))"
                 Format="N2"
                 tabIndex="4"
                 Class="mb-3" />
```

Formati disponibili:

- **Format="N0"** - Numeri interi con separatore migliaia (es: 1.234.567)
- **Format="N2"** - Numeri decimali con 2 cifre (es: 1.234,56)
- **Format="C2"** - Valuta con simbolo € (es: € 1.234,56)
- **Format="P2"** - Percentuale (es: 12,34%)

Esempio colonna numerica in DataGrid con larghezza e allineamento:

```
<PropertyColumn Property="x => x.Residenti"
                Title="Residenti"
                Format="N0">
```

```

        <HeaderStyle>
            min-width: 120px;
            text-align: right;
        </HeaderStyle>
        <CellStyle>
            min-width: 120px;
            text-align: right;
        </CellStyle>
    </PropertyColumn>

    <PropertyColumn Property="x => x.Superficie"
                    Title="Superficie (kmq)"
                    Format="N2">
        <HeaderStyle>
            min-width: 140px;
            text-align: right;
        </HeaderStyle>
        <CellStyle>
            min-width: 140px;
            text-align: right;
        </CellStyle>
    </PropertyColumn>

```

🔧 Larghezze Colonne Consigliate:

- Colonne testo corto (Sigla, Codice): **80–100px** con **text-align: center**
- Colonne testo medio (Nome, Descrizione): **180–200px**
- Colonne numeriche (int): **110–120px** con **text-align: right**
- Colonne numeriche (decimal): **130–150px** con **text-align: right**
- **IMPORTANTE:** Usare sempre **text-align: right** per colonne numeriche

🤖 Auto-Sizing Automatico delle Colonne (CONSIGLIATO):

Invece di configurare manualmente ogni colonna, usa **DataGridHelper** per calcolo automatico:

Step 1: Inizializza l'helper dopo il caricamento dati

```

@code {
    private List<Provincia> _items = new();
    private DataGridHelper<Provincia>? _gridHelper;

    private async Task LoadDataAsync()
    {
        _items = await Service.GetAllAsync();

        // Inizializza helper per auto-sizing
        if (_items.Any())
        {
            _gridHelper = new DataGridHelper<Provincia>(_items,
sampleSize: 50);
        }
    }
}

```

```
}
}
```

Step 2: Usa l'helper nelle colonne

```
@using GestioneViaggi.Services.UI

<Columns>
    @if (_gridHelper != null)
    {
        <PropertyColumn Property="x => x.Descrizione"
                        Title="Provincia"

HeaderStyle="@_gridHelper.GetColumnStyle(nameof(Provincia.Descrizione),
"Provincia")"

CellStyle="@_gridHelper.GetColumnStyle(nameof(Provincia.Descrizione),
"Provincia")" />

        <PropertyColumn Property="x => x.Residenti"
                        Title="Residenti"
                        Format="N0"

HeaderStyle="@_gridHelper.GetColumnStyle(nameof(Provincia.Residenti),
"Residenti")"

CellStyle="@_gridHelper.GetColumnStyle(nameof(Provincia.Residenti),
"Residenti")" />
    }
</Columns>
```

Vantaggi Auto-Sizing:

-  **Calcolo automatico** larghezza basato sul contenuto reale
-  **Allineamento intelligente:** numeri a destra, testo corto centrato
-  **Performance:** analizza solo primi 50 record (configurabile)
-  **Responsive:** si adatta ai dati effettivi della tabella
-  **Meno codice:** elimina configurazione manuale ripetitiva

Parametri Configurabili:

```
_gridHelper = new DataGridHelper<T>(
    items,
    sampleSize: 50,           // Numero record da analizzare (default: 50)
    minWidth: 80,             // Larghezza minima colonna (default: 80px)
    maxWidth: 400             // Larghezza massima colonna (default: 400px)
);
```

Alternativa con CellTemplate per formattazione custom:

```
<TemplateColumn Title="Residenti">
  <CellTemplate>
    @context.Item.Residenti?.ToString("N0", new
System.Globalization.CultureInfo("it-IT"))
  </CellTemplate>
</TemplateColumn>
```

⚠ IMPORTANTE:

- Culture **it-IT** è OBBLIGATORIA per avere punto come separatore migliaia
- Usare sempre **Format** o **.ToString()** con culture specificata
- Per nullable int/decimal, usare **?.ToString()** per gestire valori null

 Step 4: Creare la Page Component**Posizione:** `/Components/Pages/Tabelle/NomeEntita.razor`

```
@page "/tabelle/nomeentita"
@using Microsoft.AspNetCore.Authorization
@using GestioneViaggi.Models
@using GestioneViaggi.Services.CRUD
@using MudBlazor
@inject NomeEntitaService Service
@inject IDialogService DialogService
@inject ISnackbar Snackbar
@attribute [Authorize]

@if (!string.IsNullOrEmpty(_errorMessage))
{
  <MudAlert Severity="Severity.Error" Variant="Variant.Filled"
Class="my-2">
    <MudText Typo="Typo.h6">ERRORE:</MudText>
    <MudText>@_errorMessage</MudText>
  </MudAlert>
}
else
{
  <EnterpriseDataGrid T="NomeEntita"
    Items="@_items"
    Title="Gestione [Nome Tabella]"
    SearchFunction="@Search"
    @bind-SelectedItem="_selectedItem">

    <ToolBarContent>
      <MudButton Variant="Variant.Filled"
        Color="Color.Primary"
        StartIcon="@Icons.Material.Filled.Add"
```

```

                OnClick="@OpenCreateDialog"
                Class="ml-2">
                    Nuovo
                </MudButton>
            </ToolBarContent>

            <Columns>
                <PropertyColumn Property="x => x.Campo1" Title="Campo 1" />
                <PropertyColumn Property="x => x.Campo2" Title="Campo 2" />

                <EnterpriseActionsColumn T="NomeEntita"
                    OnEdit="@OpenEditDialog"
                    OnDelete="@DeleteItem" />
            </Columns>

        </EnterpriseDataGrid>
    }

    @code {
        private List<NomeEntita> _items = new();
        private NomeEntita? _selectedItem;
        private string _errorMessage = string.Empty;

        protected override async Task OnInitializedAsync()
        {
            await LoadDataAsync();
        }

        private async Task LoadDataAsync()
        {
            try
            {
                _errorMessage = string.Empty;
                _items = await Service.GetAllAsync();
            }
            catch (Exception ex)
            {
                _errorMessage = $"Errore durante il caricamento:
{ex.Message}";
            }
        }

        private bool Search(NomeEntita item, string searchString)
        {
            if (string.IsNullOrEmpty(searchString)) return true;
            if (item.Campo1?.Contains(searchString,
StringComparison.OrdinalIgnoreCase) == true) return true;
            if (item.Campo2?.Contains(searchString,
StringComparison.OrdinalIgnoreCase) == true) return true;
            return false;
        }

        private async Task OpenCreateDialog()
        {

```

```
var parameters = new DialogParameters<NomeEntitaDialog>
{
    { x => x.Entity, new NomeEntita() },
    { x => x.IsEditMode, false }
};

var options = new DialogOptions
{
    CloseButton = true,
    MaxWidth = MaxWidth.Small,
    FullWidth = true
};

var dialog = await DialogService.ShowAsync<NomeEntitaDialog>
("Nuovo", parameters, options);
var result = await dialog.Result;

if (!result!.Canceled && result.Data is NomeEntita newItem)
{
    try
    {
        var created = await Service.CreateAsync(newItem);
        _items.Add(created);
        Snackbar.Add("Creato con successo", Severity.Success);
        StateHasChanged();
    }
    catch (Exception ex)
    {
        Snackbar.Add($"Errore: {ex.Message}", Severity.Error);
    }
}

private async Task OpenEditDialog(NomeEntita item)
{
    var parameters = new DialogParameters<NomeEntitaDialog>
    {
        { x => x.Entity, new NomeEntita { Id = item.Id, Campo1 =
item.Campo1, Campo2 = item.Campo2 } },
        { x => x.IsEditMode, true }
    };

    var options = new DialogOptions
    {
        CloseButton = true,
        MaxWidth = MaxWidth.Small,
        FullWidth = true
    };

    var dialog = await DialogService.ShowAsync<NomeEntitaDialog>
("Modifica", parameters, options);
    var result = await dialog.Result;

    if (!result!.Canceled && result.Data is NomeEntita updatedItem)
```

```
{
    try
    {
        var updated = await Service.UpdateAsync(updatedItem);
        var index = _items.FindIndex(c => c.Id == updated.Id);
        if (index >= 0)
        {
            _items[index] = updated;
        }
        Snackbar.Add("Aggiornato con successo", Severity.Success);
        StateHasChanged();
    }
    catch (Exception ex)
    {
        Snackbar.Add($"Errore: {ex.Message}", Severity.Error);
    }
}

private async Task DeleteItem(NomeEntita item)
{
    var parameters = new DialogParameters
    {
        { "Title", "Attenzione" },
        { "ContentText", $"Vuoi veramente cancellare questo record? ({item.Campo1})" }
    };

    var options = new DialogOptions { CloseButton = true, MaxWidth =
MaxWidth.ExtraSmall };

    var dialog = await
DialogService.ShowAsync<DeleteConfirmationDialog>("Delete", parameters,
options);
    var result = await dialog.Result;

    if (!result!.Canceled)
    {
        try
        {
            var success = await Service.DeleteAsync(item.Id);
            if (success)
            {
                _items.Remove(item);
                Snackbar.Add("Eliminato con successo",
Severity.Success);
                StateHasChanged();
            }
            else
            {
                Snackbar.Add("Impossibile eliminare",
Severity.Warning);
            }
        }
    }
}
```

```
        catch (Exception ex)
        {
            Snackbar.Add($"Errore: {ex.Message}", Severity.Error);
        }
    }
}
```

Step 5: Registrare il Service

Posizione: `/MauiProgram.cs`

Aggiungi nella sezione `// CRUD SERVICES`:

```
builder.Services.AddScoped<NomeEntitaService>();
```

Esempio Completo: ana_geo_capoluogo

File creati:

1.  `/Models/BaseEntity.cs` - Classe base con Id
2.  `/Models/Capoluogo.cs` - Model specifico
3.  `/Services/CRUD/ICrudService.cs` - Interfaccia generica
4.  `/Services/CRUD/BaseCrudService.cs` - Service base con GetAll, GetByld, Delete
5.  `/Services/CRUD/CapoluogoService.cs` - Service specifico con Create, Update, MapFromReader
6.  `/Components/Shared/CapoluogoDialog.razor` - Dialog modale
7.  `/Components/Pages/Tabelle/Capoluoghi.razor` - Pagina completa con DataGrid

Registrazione in MauiProgram.cs:

```
builder.Services.AddScoped<CapoluogoService>();
```

Styling

Il sistema usa automaticamente il foglio di stile:

- `/wwwroot/css/premium-saas-ULTRA-SPECIFIC.css`

Il componente `EnterpriseDataGrid` applica automaticamente:

- Dark/Light mode support
- Hover effects (Edit: giallo, Delete: rosso)

- Typography: Headers 12px uppercase, Celle 14px
 - Striping e dense mode disabilitati per controllo CSS completo
-

✅ Checklist per Nuova Tabella

- ☐ Creare Model in `/Models/`
 - ☐ Creare Service in `/Services/CRUD/`
 - ☐ Implementare `CreateAsync()`, `UpdateAsync()`, `MapFromReader()`
 - ☐ Creare Dialog in `/Components/Shared/`
 - ☐ Creare Page in `/Components/Pages/Tabelle/`
 - ☐ Registrare Service in `MauiProgram.cs`
 - ☐ Testare: Create, Read, Update, Delete
 - ☐ Verificare validazione form
 - ☐ Testare funzione Search
-



Note Importanti

1. **ID sempre nascosto:** Il campo `Id` da `BaseEntity` non va mai mostrato nelle colonne del DataGrid
 2. **Validazione:** Usa `DataAnnotations` nel Model per validazione automatica
 3. **Nullable:** Usa `(object?)value ?? DBNull.Value` per parametri nullable
 4. **Error handling:** Sempre try-catch con Snackbar per feedback utente
 5. **StateHasChanged:** Chiamare dopo operazioni CRUD per refresh UI
-



Build e Test

```
dotnet build -f net9.0-maccatalyst
```

✅ **Build Status:** Completata con successo (0 errori)